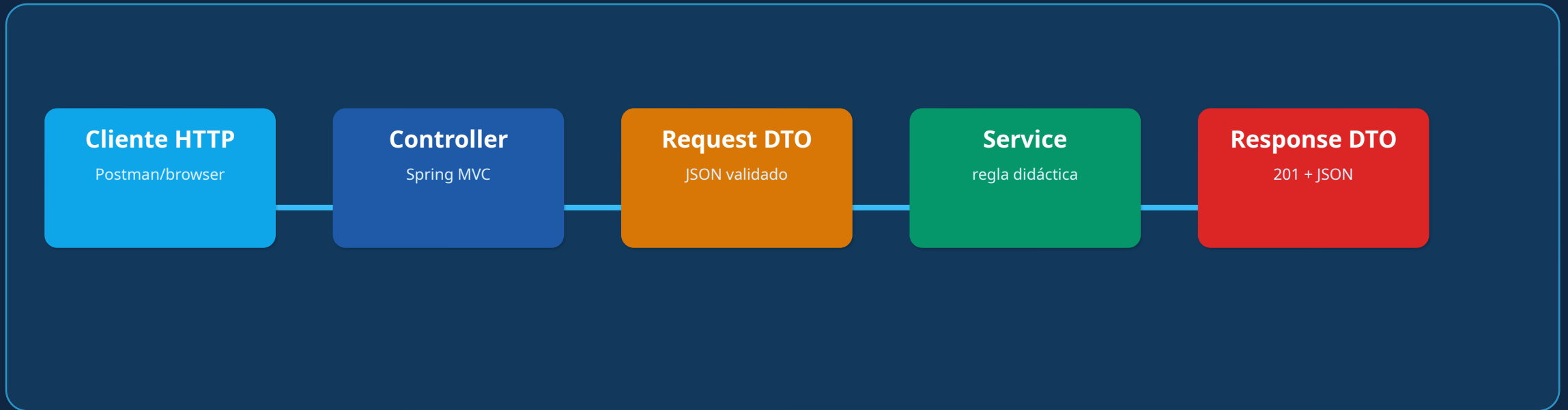


Flujo completo de una petición HTTP

Spring MVC real · DTO · @Valid · Service · ResponseEntity · JSON



Objetivo docente: que el estudiante pueda dibujar, narrar y defender qué ocurre desde el POST hasta la respuesta.

01

Parte 1 · Qué representa el dibujo

Antes de escribir código, los estudiantes deben comprender el recorrido completo.

Cliente HTTP -> Controller -> DTO validado -> Service -> Dominio -> Response DTO -> Cliente HTTP

El dibujo no es decoración: es el contrato mental del backend

La pizarra debe mostrar responsabilidades, no solo flechas.

VISIÓN GENERAL

Lo que entra

Una petición HTTP con verbo, URL, headers y body JSON.

El cliente no llama métodos Java: envía mensajes HTTP.

El contrato externo se expresa con DTOs y códigos HTTP.

Lo que procesa

Spring MVC enruta la petición al método correcto.

@RequestBody transforma JSON en un objeto Java.

@Valid ejecuta reglas de formato antes del caso de uso.

Lo que sale

Response DTO define el JSON de respuesta.

ResponseEntity define status y body.

201 CREATED comunica que se creó un recurso.

El flujo completo en una sola lectura

Esta es la versión que el docente puede dibujar en la pizarra.

PIZARRA

Cliente HTTP

POST /api/clientes + JSON

ClienteController

@RequestBody + @Valid

CrearClienteRequest

DTO de entrada validado

ClienteService

crea objeto de dominio

Cliente

modelo de dominio

ClienteResponse

DTO de salida

Cliente HTTP

201 CREATED + JSON

Lectura didáctica: cada bloque tiene una responsabilidad distinta. Si una pieza hace el trabajo de otra, la arquitectura empieza a deformarse.

02 Parte 2 · Entrada HTTP y Spring MVC

Ahora se explica cada bloque del recorrido, desde el cliente hacia el backend.

```
POST /api/clientes  
Content-Type: application/json  
{ ... }
```

Cliente HTTP: Postman, navegador o IntelliJ HTTP Client

El backend no sabe si la petición vino de Postman, React o una app móvil.

BLOQUE 1

Qué debe enseñar

Un cliente HTTP prepara una solicitud hacia una URL concreta.

POST se usa cuando se quiere crear un recurso nuevo.

El body viaja normalmente como JSON.

Content-Type indica al servidor cómo interpretar el cuerpo.

El estudiante debe diferenciar request y response.

```
POST http://localhost:8080/api/clientes
Content-Type: application/json
```

```
{
  "documento": "7845123",
  "nombreCompleto": "Ana Torres",
  "email": "ana@correo.com",
  "telefono": "+591 70000001"
}
```

ClienteController: la puerta de entrada HTTP

El Controller traduce HTTP hacia una llamada Java controlada.

BLOQUE 2

```
@RestController
@RequestMapping("/api/clientes")
public class ClienteController {

    @PostMapping
    public ResponseEntity<ClienteResponse> crear(
        @Valid @RequestBody CrearClienteRequest request) {

        Cliente creado = clienteService.registrar(...);

        return ResponseEntity.status(HttpStatus.CREATED)
            .body(toResponse(creado));
    }
}
```

Qué justificar en clase

@RestController convierte retornos Java en JSON.

@RequestMapping define la ruta base del recurso.

@PostMapping asocia el método al verbo POST.

El Controller no debe contener reglas de negocio complejas.

Su trabajo es recibir, delegar y responder.

@RequestBody + @Valid: dos responsabilidades diferentes

Uno transforma datos; el otro valida el contrato de entrada.

BLOQUE 3

@RequestBody

Convierte el JSON recibido en el body de la petición en una instancia Java. Spring usa el conversor JSON configurado por defecto para deserializar los campos.

```
@RequestBody CrearClienteRequest request
```

Sin @RequestBody, Spring no sabría que debe leer el cuerpo JSON para construir el DTO.

@Valid

Activa las anotaciones de Jakarta Validation colocadas sobre el DTO. Si el contrato no se cumple, la petición se rechaza antes de ejecutar la lógica principal.

```
@Valid @RequestBody CrearClienteRequest request
```

Sin @Valid, @NotBlank, @Email, @Size y @Pattern pueden estar escritos, pero no se ejecutan en ese punto.

CrearClienteRequest: DTO de entrada

El DTO de entrada protege el contrato externo y evita exponer el dominio.

BLOQUE 4

```
public record CrearClienteRequest(  
    @NotBlank(message = "El documento es obligatorio")  
    @Size(max = 30)  
    String documento,  
  
    @NotBlank  
    @Size(max = 180)  
    String nombreCompleto,  
  
    @NotBlank  
    @Email  
    String email,  
  
    @Pattern(regexp = "[0-9+\\- ]{7,20}$")  
    String telefono  
) {}
```

Puntos clave

Request DTO representa lo que el cliente puede enviar.

No debe ser la entidad de dominio ni la entidad JPA.

Las validaciones de formato viven cerca del contrato HTTP.

El record declara componentes en la cabecera, no campos dentro del cuerpo.

03 Parte 3 · Procesamiento interno

El backend deja de hablar HTTP y pasa a trabajar con objetos de aplicación y dominio.

Controller -> Service -> Cliente

ClienteService: lógica de aplicación en memoria

En Capítulo 04 se usa memoria para no mezclar REST con JPA todavía.

BLOQUE 5

```
@Service
public class ClienteService {

    private final List<Cliente> clientes = new ArrayList<>();
    private final AtomicLong secuencia = new AtomicLong(0);

    public Cliente registrar(String documento, String nombreCompleto,
                           String email, String telefono) {
        Cliente cliente = new Cliente(
            secuencia.incrementAndGet(), documento,
            nombreCompleto, email, telefono
        );
        clientes.add(cliente);
        return cliente;
    }
}
```

Justificación pedagógica

La lista en memoria permite aprender MVC sin persistencia.

El Service concentra la operación registrar.

El Controller no decide cómo guardar ni cómo crear IDs.

Más adelante este Service evolucionará hacia casos de uso y puertos.

Cliente: modelo de dominio mínimo

El dominio representa el concepto del negocio, no el formato HTTP.

BLOQUE 6

Qué es dominio

Cliente es una idea del negocio ParkFlow 360.

No debería depender de @RestController, JSON ni ResponseEntity.

En este capítulo todavía no se usa @Entity.

Debe poder explicarse sin hablar de Postman ni PostgreSQL.

```
public class Cliente {  
    private final Long id;  
    private final String documento;  
    private final String nombreCompleto;  
    private final String email;  
    private final String telefono;  
}
```

Frase para la pizarra

El dominio no nace para satisfacer al framework; nace para representar conceptos y reglas del problema.

04 Parte 4 · Respuesta HTTP profesional

La salida también debe tener contrato: DTO + status code + JSON.

Cliente -> ClienteResponse -> 201 CREATED + JSON

ClienteResponse: DTO de salida

No todo lo que existe internamente debe salir por la API.

BLOQUE 7

```
public record ClienteResponse(  
    Long id,  
    String documento,  
    String nombreCompleto,  
    String email,  
    String telefono  
) {}
```

Regla docente

Request DTO define lo que entra. Response DTO define lo que sale. El dominio no debe ser el contrato externo de la API.

Por qué no devolver Cliente directamente

El response representa el contrato público de salida.

Permite ocultar atributos internos o sensibles.

Evita que el contrato externo cambie cada vez que cambia el dominio.

Prepara a los estudiantes para separar dominio, persistencia y web.

ResponseEntity + 201 CREATED

El código HTTP también forma parte del contrato de la API.

BLOQUE 8

```
return ResponseEntity  
    .status(HttpStatus.CREATED)  
    .body(toResponse(creado));
```

Qué comunica 201

El servidor aceptó la petición.

Se creó un nuevo recurso.

El body contiene la representación creada.

Comparación clave

Código	Cuándo usarlo
200 OK	Consulta exitosa
201 CREATED	Creación exitosa
400 BAD REQUEST	Entrada inválida
404 NOT FOUND	Recurso no existe

05 Parte 5 · Cómo defender el flujo

El estudiante debe narrar el recorrido sin leer el código.

¿Qué entra? ¿Quién lo recibe? ¿Quién valida? ¿Quién procesa? ¿Qué sale?

Guion oral para explicar el flujo en clase

Úselo como libreto de defensa individual.

DEFENSA

Respuesta esperada del estudiante

1. El cliente HTTP envía un POST a /api/clientes con un body JSON y Content-Type application/json.
2. Spring MVC localiza ClienteController gracias a @RestController, @RequestMapping y @PostMapping.
3. @RequestBody convierte el JSON en CrearClienteRequest.
4. @Valid ejecuta las reglas del DTO: @NotBlank, @Size, @Email y @Pattern.
5. El Controller delega en ClienteService; no resuelve la lógica principal en el método HTTP.
6. ClienteService crea un objeto Cliente de dominio y lo guarda temporalmente en memoria.
7. El Controller transforma el Cliente en ClienteResponse.
8. ResponseEntity devuelve 201 CREATED y un JSON de respuesta al cliente HTTP.

Errores que debe detectar el estudiante

Estos errores demuestran que el flujo no fue comprendido.

ERRORES

Devolver Cliente directamente

Rompe separación entre dominio y contrato externo.

Poner lógica compleja en Controller

El adaptador web empieza a absorber negocio.

Olvidar @Valid

Las anotaciones de validación no se ejecutan.

Usar JPA en este capítulo

Mezcla persistencia antes de dominar HTTP/MVC.

Usar siempre 200

Pierde semántica HTTP y confunde al cliente.

Confundir Request DTO con Response DTO

Se pierde claridad entre entrada y salida.

Preguntas de defensa sobre el diagrama

Sirven para comprobar si el estudiante entiende realmente el recorrido.

PREGUNTAS

Banco rápido

1. ¿Qué diferencia hay entre el cliente HTTP y el Cliente de dominio?
2. ¿Qué hace @RequestBody exactamente?
3. ¿Qué ocurre si el DTO tiene @NotBlank pero falta @Valid?
4. ¿Por qué CrearClienteRequest no debería ser la misma clase que Cliente?
5. ¿Por qué ClienteResponse no debería ser la entidad de dominio completa?
6. ¿Qué responsabilidad tiene ClienteController?
7. ¿Qué responsabilidad tiene ClienteService en este capítulo?
8. ¿Por qué POST exitoso debería responder 201 y no simplemente 200?
9. ¿Qué parte del flujo se reemplazará cuando entremos a JPA/PostgreSQL?
10. ¿Cómo explicarías el recorrido completo en menos de un minuto?

Cierre: la frase que deben memorizar y comprender

El objetivo no es aprender anotaciones sueltas, sino entender el flujo.

CIERRE

Una petición HTTP entra como JSON, Spring MVC la convierte en DTO, Jakarta Validation valida el contrato, el Controller delega en el Service, el dominio representa el negocio y ResponseEntity devuelve un status HTTP coherente con un JSON de respuesta.

